



HTTPX

Test Suite passing pypi package 0.28.1

A next-generation HTTP client for Python.

HTTPX is a fully featured HTTP client for Python 3, which provides sync and async APIs, and support for both HTTP/1.1 and HTTP/2.

Install HTTPX using pip:

```
$ pip install httpx
```

Now, let's get started:

```
>>> import httpx
>>> r = httpx.get('https://www.example.org/')
>>> r
<Response [200 OK]>
>>> r.status_code
200
>>> r.headers['content-type']
'text/html; charset=UTF-8'
>>> r.text
'<!doctype html>\n<html>\n<head>\n<title>Example Domain</title>...'
```

Or, using the command-line client.

```
# The command line client is an optional dependency.
$ pip install 'httpx[cli]'
```

Which now allows us to use HTTPX directly from the command-line...

```
$ httpx --help

HTTPX 🦋

A next generation HTTP client.

Usage: httpx <URL> [OPTIONS]

-m, --method METHOD      Request method, such as GET, POST, PUT, PATCH, DELETE, OPTIONS, HEAD.
                        [Default: GET, or POST if a request body is included]

-p, --params <NAME VALUE> ... Query parameters to include in the request URL.

-c, --content TEXT       Byte content to include in the request body.

-d, --data <NAME VALUE> ... Form data to include in the request body.

-f, --files <NAME FILENAME> ... Form files to include in the request body.

-j, --json TEXT          JSON data to include in the request body.

-h, --headers <NAME VALUE> ... Include additional HTTP headers in the request.

--cookies <NAME VALUE> ... Cookies to include in the request.

--auth <USER PASS>       Username and password to include in the request. Specify '-' for the password to use a
                        password prompt. Note that using --verbose/-v will expose the Authorization header,
                        including the password encoding in a trivially reversible format.

--proxy URL              Send the request via a proxy. Should be the URL giving the proxy address.

--timeout FLOAT          Timeout value to use for network operations, such as establishing the connection,
                        reading some data, etc... [Default: 5.0]

--follow-redirects       Automatically follow redirects.

--no-verify              Disable SSL verification.

--http2                  Send the request using HTTP/2, if the remote server supports it.

--download FILE          Save the response content as a file, rather than displaying it.

-v, --verbose            Verbose output. Show request as well as response.

--help                  Show this message and exit.

$
```

Sending a request...

```
httpx
$ httpx http://httpbin.org/json
HTTP/1.1 200 OK
Date: Mon, 13 Sep 2021 09:59:51 GMT
Content-Type: application/json
Content-Length: 429
Connection: keep-alive
Server: gunicorn/19.9.0
Access-Control-Allow-Origin: *
Access-Control-Allow-Credentials: true

{
  "slideshow": {
    "author": "Yours Truly",
    "date": "date of publication",
    "slides": [
      {
        "title": "Wake up to WonderWidgets!",
        "type": "all"
      },
      {
        "items": [
          "Why <em>WonderWidgets</em> are great",
          "Who <em>buys</em> WonderWidgets"
        ],
        "title": "Overview",
        "type": "all"
      }
    ],
    "title": "Sample Slide Show"
  }
}
```

Features

HTTPX builds on the well-established usability of `requests`, and gives you:

- A broadly [requests-compatible API](#).
- Standard synchronous interface, but with [async support](#) if you need it.
- HTTP/1.1 [and HTTP/2 support](#).
- Ability to make requests directly to [WSGI applications](#) or [ASGI applications](#).
- Strict timeouts everywhere.
- Fully type annotated.
- 100% test coverage.

Plus all the standard features of `requests` ...

- International Domains and URLs
- Keep-Alive & Connection Pooling
- Sessions with Cookie Persistence
- Browser-style SSL Verification
- Basic/Digest Authentication
- Elegant Key/Value Cookies
- Automatic Decompression
- Automatic Content Decoding

- Unicode Response Bodies
- Multipart File Uploads
- HTTP(S) Proxy Support
- Connection Timeouts
- Streaming Downloads
- .netrc Support
- Chunked Requests

Documentation

For a run-through of all the basics, head over to the [QuickStart](#).

For more advanced topics, see the **Advanced** section, the [async support](#) section, or the [HTTP/2](#) section.

The [Developer Interface](#) provides a comprehensive API reference.

To find out about tools that integrate with HTTPX, see [Third Party Packages](#).

Dependencies

The HTTPX project relies on these excellent libraries:

- `httpcore` - The underlying transport implementation for `httpx`.
- `h11` - HTTP/1.1 support.
- `certifi` - SSL certificates.
- `idna` - Internationalized domain name support.
- `sniffio` - Async library autodetection.

As well as these optional installs:

- `h2` - HTTP/2 support. (*Optional, with `httpx[http2]`*)
- `socksio` - SOCKS proxy support. (*Optional, with `httpx[socks]`*)
- `rich` - Rich terminal support. (*Optional, with `httpx[cli]`*)
- `click` - Command line client support. (*Optional, with `httpx[cli]`*)
- `brotli` or `brotlicffi` - Decoding for "brotli" compressed responses. (*Optional, with `httpx[brotli]`*)
- `zstandard` - Decoding for "zstd" compressed responses. (*Optional, with `httpx[zstd]`*)

A huge amount of credit is due to `requests` for the API layout that much of this work follows, as well as to `urllib3` for plenty of design inspiration around the lower-level networking details.

Installation

Install with pip:

```
$ pip install httpx
```

Or, to include the optional HTTP/2 support, use:

```
$ pip install httpx[http2]
```

To include the optional brotli and zstandard decoders support, use:

```
$ pip install httpx[brotli,zstd]
```

HTTPX requires Python 3.9+